CampiFi — Complete System Architecture & Code Guide

A Step-by-Step Technical Reference of Every Class, File, Workflow, and Database Operation

Project Domain: Local Campus Network Chat	**Backend:** Spring Boot 3.2.5 (Java 17/21)
Protocol: WebSockets + STOMP over SockJS	**Database:** MySQL 8.4 via Spring Data JPA / Hibernate
Network Scope: Campus Subnet (10.0.0.0/8 / Hotspot)	**Security:** SHA-256 PIN Hashing + Dynamic Role RBAC

1. Executive Summary & Runtime Flow

CampiFi is an off-grid campus communication platform engineered to function without external internet connectivity. It operates over the local Wi-Fi or mobile hotspot. Devices physically within the campus IP range can connect, communicate in real time, and organize into academic hierarchies without relying on commercial cloud messaging platforms like WhatsApp or Discord.

End-to-End Runtime Execution Order:

- **Step 1: Application Boot:** Spring Boot starts, reads application.properties, connects to MySQL via JDBC, validates tables via Hibernate DDL-auto, and runs DataInitializer to auto-seed initial networks.
- **Step 2: Gatekeeper IP Check:** A device opens http://<server-ip>:8080/. PageController calls NetworkUtils.isHostDevice(). The host computer is routed to admin.html; other campus devices are routed to index.html.
- **Step 3: Frictionless Authentication:** The student enters their institutional email (e.g. vikram_choudhury@srmap.edu.in). AuthController calls UserService.findOrCreate() which parses the display name, creates a User entity, and stores an 8-hour UUID session in SessionService.
- **Step 4: Network Directory:** The client is redirected to channels.html. It fetches all root channels from GET /api/channels. Public networks can be viewed immediately; Private networks prompt for a 4-digit PIN.
- **Step 5: PIN Verification:** Submitting a 4-digit PIN triggers POST /api/channels/{id}/verify-pin. ChannelService hashes the PIN with **SHA-256** and compares it against the database hash. On success, the user is joined as a ChannelMember.
- **Step 6: Real-Time Chat Connection:** The user opens a channel or sub-channel (chat.html?room=...). The client creates a **SockJS + STOMP** WebSocket connection to /ws, subscribes to /topic/room/{roomName}, and broadcasts a JOIN message.
- **Step 7: Bi-Directional Messaging:** When any user types a message and clicks Send, it sends a STOMP payload to /app/chat/{room}. ChatController intercepts it, attaches server timestamps, and publishes it to all active subscribers via /topic/room/{room} instantaneously.

2. Application Boot & Configuration Layer

Class / File: HotspotChatApplication.java
Path: src/main/java/com/hotspotchat/HotspotChatApplication.java

The entry point of the Spring Boot application. Decorated with @SpringBootApplication and @EnableScheduling. Initializes the Spring ApplicationContext, component scanning across all sub-packages, and launches the embedded Tomcat server on port 8080.

Class / File: application.properties
Path: src/main/resources/application.properties

Central configuration file. Configures server port (8080), network bind address (0.0.0.0 for LAN access), MySQL JDBC datasource URL (jdbc:mysql://localhost:3306/campifi), credentials (campifi_user/campifi_pass), Hibernate dialect, spring.jpa.hibernate.ddl-auto=update (auto-creates/updates tables without data loss), and upload size caps (50MB).

Class / File: DataInitializer.java
Path: src/main/java/com/hotspotchat/DataInitializer.java

Implements CommandLineRunner to execute immediately after application startup. Checks if the database already has channels; if empty, it auto-seeds 4 standard networks: *University Announcements* (Public), *Campus Events & Sports* (Public), *Computer Science Dept* (Private, PIN 1234), and *Faculty Council* (Private, PIN 9999). Also pre-seeds sub-channels (e.g. 3rd Year Core, Lab Group A).

3. Database Schema & JPA Entity Layer

Spring Data JPA maps these Java classes directly to MySQL tables using annotations like @Entity, @Table, @Id, @ManyToOne, and @OneToMany. You do not need to write raw SQL table definitions; Hibernate generates them.

User.java (Table: users)

Represents a registered campus participant. Fields: id (Primary Key), email (unique index), displayName (derived from email), role (Enum: USER or ADMIN), createdAt. Drives privilege authorization across the entire platform.

Channel.java (Table: channels)

Represents a root network. Fields: id, name, description, type (Enum: PUBLIC or PRIVATE), pinHash (stores the SHA-256 hash of the 4-digit PIN for private networks, null for public), createdAt. Has a @OneToMany relationship with SubChannel and ChannelMember.

SubChannel.java (Table: sub_channels)

Represents nested sub-groups inside a channel. Fields: id, name, description, channel (@ManyToOne foreign key back to parent Channel). Allows dividing large departments into smaller batch/lab groups.

ChannelMember.java (Table: channel_members)

Association entity representing channel membership. Fields: id, user (FK), channel (FK), isAdmin (boolean flag indicating if the member is a Master Admin of this channel), isBanned (boolean flag).

SubChannelAdmin.java (Table: sub_channel_admins)

Dedicated mapping entity that binds a specific User to a specific SubChannel as an administrator. Allows assigning faculty leads to individual sub-groups without making them global root channel admins.

ChatMessage.java (DTO Model)

In-memory Data Transfer Object used by WebSockets (not mapped to DB). Fields: type (CHAT, JOIN, LEAVE), sender, content, timestamp, room.

4. Data Access Layer (Spring Data Repositories)

All repositories reside in com.hotspotchat.repository and extend JpaRepository<Entity, ID>. Spring dynamically generates optimized SQL queries from method names at runtime.

UserRepository.java

• findByEmail(String email): Retrieves user by email.
• existsByEmail(String email): Fast existence verification.

ChannelRepository.java

• findByName(String name): Look up network by name.
• existsByName(String name): Validates unique channel naming.

SubChannelRepository.java

• findByChannelId(Long channelId): Returns all sub-channels under a specific root network.
• deleteByChannelId(Long id): Cascade deletion helper.

ChannelMemberRepository.java

• findByUserAndChannel(User, Channel): Checks user membership in a channel.
• existsByUserAndIsAdminTrue(User): Crucial for role demotion logic—checks if user is a Master Admin in *any* channel.
• findByChannelAndIsBannedFalse(Channel): Retrieves active non-banned members.

SubChannelAdminRepository.java

• findBySubChannelId(Long id): Retrieves all admins assigned to a sub-channel.
• existsByUserAndSubChannel(User, SubChannel): Checks specific admin assignment.
• existsByUser(User): Checks if a user is an admin of *any* sub-channel anywhere.

5. Network Gatekeeper, Authentication & Sessions

NetworkUtils.java
Path: src/main/java/com/hotspotchat/service/NetworkUtils.java

Validates the physical network location of connecting devices.
• isHostDevice(HttpServletRequest request): Inspects request.getRemoteAddr() against 127.0.0.1, ::1, and all local network interface addresses of the server. Only the physical computer running the Spring Boot server qualifies as the Host.
• isCampusIp(String ip): Verifies that the client IP starts with 10. (campus 10.0.0.0/8 network) or is a local loopback.

PageController.java
Path: src/main/java/com/hotspotchat/controller/PageController.java

The root HTTP router for GET /. Uses NetworkUtils.isHostDevice(). If accessed from the server machine itself, it redirects to admin.html (Network Management Dashboard). If accessed from any other client device on the Wi-Fi, it directs to index.html (User Login).

UserService.java
Path: src/main/java/com/hotspotchat/service/UserService.java

Handles user lifecycle.
• findOrCreate(email): Finds an existing user or creates a new one.
• parseDisplayName(email): Extracts human-readable names from institutional email addresses (e.g. harsh.jha23@uni.edu → **Harsh Jha**).
• promoteToAdmin(email) and demoteToUser(email): Toggles global user roles.

SessionService.java
Path: src/main/java/com/hotspotchat/service/SessionService.java

High-performance in-memory session manager using a ConcurrentHashMap<String, SessionData>.
• When a user logs in, generates a cryptographically random UUID token (e.g. a7f8e3...) returned to the client and stored in sessionStorage.
• Each session stores userId, email, displayName, role, and an 8-hour expiry timestamp.
• All REST APIs authenticate callers via the X-Session-Token HTTP header.

AuthController.java
Path: src/main/java/com/hotspotchat/controller/AuthController.java

REST controller for user access.
• POST /api/auth/login: Accepts { email }, validates domain format, calls findOrCreate(), creates session token.
• GET /api/auth/check-session: Validates token validity from X-Session-Token header.
• POST /api/auth/logout: Invalidates the token.

6. Channel & Admin Management Business Logic

ChannelService.java
Path: src/main/java/com/hotspotchat/service/ChannelService.java

The core business brain of CampiFi. Manages all channel operations:
• createChannel(name, desc, type, pin): Creates root network. If type is PRIVATE, hashes the 4-digit PIN using SHA-256 before saving.
• hashPin(pin): Computes MessageDigest.getInstance("SHA-256"). Raw PINs are never stored in the database.
• verifyPin(channelId, inputPin): Hashes input and compares it with channel.getPinHash().
• joinChannel(channelId, user, pin): Validates PIN if private, ensures user is not banned, creates ChannelMember.
• addChannelAdmin(channelId, email): Promotes user to Master Admin for that channel and elevates global role to ADMIN.
• removeChannelAdmin(channelId, userId): Sets ChannelMember.isAdmin = false and triggers downgradeRoleIfNoAdminRoles().
• addSubChannelAdmin(subChannelId, email): Saves SubChannelAdmin record, elevates user role to ADMIN.
• removeSubChannelAdmin(subChannelId, userId): Deletes admin record and triggers downgradeRoleIfNoAdminRoles().
• **downgradeRoleIfNoAdminRoles(User user) (Critical Bugfix):** Verifies if user is still a Master Admin anywhere (existsByUserAndIsAdminTrue) OR still a sub-channel admin anywhere (existsByUser). If both are false, safely revokes global role back to USER.

AdminChannelController.java
Path: src/main/java/com/hotspotchat/controller/AdminChannelController.java

Host-only REST endpoints under /api/admin/**. Every endpoint strictly checks networkUtils.isHostDevice(request), returning HTTP 403 Forbidden to any client trying to call admin APIs remotely.
• POST /api/admin/channels: Create network.
• DELETE /api/admin/channels/{id}: Delete network.
• POST /api/admin/channels/{id}/sub-channels: Create sub-channel.
• POST /api/admin/channels/{id}/admins: Assign Master Admin.
• DELETE /api/admin/channels/{id}/admins/{uid}: Remove Master Admin.
• POST /api/admin/sub-channels/{id}/admins: Assign Sub-Channel Admin.
• DELETE /api/admin/sub-channels/{id}/admins/{uid}: Remove Sub-Channel Admin.

ChannelController.java
Path: src/main/java/com/hotspotchat/controller/ChannelController.java

Student/client REST endpoints under /api/channels/**. Requires valid X-Session-Token header.
• GET /api/channels: Returns all network summaries.
• POST /api/channels/{id}/verify-pin: Checks PIN.
• POST /api/channels/{id}/join: Joins channel.
• GET /api/channels/{id}/sub-channels: Returns sub-channels (verifies membership for private channels).
• POST /api/channels/{id}/leave: Leaves channel.

7. Real-Time WebSocket & STOMP Messaging Engine

WebSocketConfig.java
Path: src/main/java/com/hotspotchat/config/WebSocketConfig.java

Annotated with @Configuration and @EnableWebSocketMessageBroker. Implements WebSocketMessageBrokerConfigurer.
• configureMessageBroker(): Enables an in-memory STOMP message broker on prefix /topic. Sets application destination prefix to /app.
• registerStompEndpoints(): Registers the handshake endpoint /ws. Adds .withSockJS() so that if standard WebSockets are blocked by network firewalls or older browsers, it gracefully falls back to XHR streaming or long-polling.

ChatController.java
Path: src/main/java/com/hotspotchat/controller/ChatController.java

WebSocket STOMP message handler.
• @MessageMapping("/chat/{room}"): Receives messages sent by clients to /app/chat/{room}.
• @SendTo("/topic/room/{room}"): Broadcasts the processed message to all clients subscribed to that room.
• Attaches server-side timestamp formatted as HH:mm:ss to prevent client clock tampering.

8. Frontend Architecture & User Interface

index.html (Client Connect)

Clean, modern entry page. Contains single institution email input field. On submit, calls /api/auth/login, stores the session token in sessionStorage, and redirects the student to channels.html.

channels.html (Network Directory)

Displays responsive cards for all campus networks. Shows name, member count, public/private badge. Clicking a private network opens a 4-box PIN modal. Once joined, an accordion panel slides open displaying all available sub-networks with a 'Chat' launch button.

admin.html (Host Management Dashboard)

Dedicated host server dashboard. Shows all networks with toggle arrows (■/▼) expanding child sub-channels. Features 'Admins' button on each network and sub-channel opening an admin assignment modal to add/remove professors by institutional email.

chat.html (Live Chat Room)

The real-time conversation screen. Features topbar with channel name and live member count, an active member presence strip, scrollable chat message container with distinct sender bubbles, and bottom text input.

app.js (Unified Client Logic)

Drives all client behavior in ~900 lines of structured vanilla JS:
• initHomepage(): Login form handling.
• initChannelsPage(): Renders network cards, PIN modal logic, sub-channel expanders.
• initAdminDashboard(): Network CRUD, accordion toggling, admin assignment modal.
• initChat(room, user): Connects SockJS + STOMP, subscribes to /topic/room/{room}, renders messages, tracks presence.

style.css (Design System)

Comprehensive dark-mode stylesheet. Defines CSS variables for background (#0d1117), surface (#161b22), accent purple (#8957e5), green (#238636), and red (#da3633). Responsive grid layouts, keyframe animations, and modal overlays.

9. Key Technical Viva & Presentation Q&A;

Q1: Why use WebSockets instead of normal HTTP polling?

Answer: HTTP polling creates massive overhead because the client must constantly send new HTTP request headers every few seconds. WebSockets establish a single persistent full-duplex TCP connection. Once the handshake is complete, messages travel with negligible header overhead (<6 bytes) and are pushed instantly from the server without waiting for client requests.

Q2: What is STOMP and why do we need it over raw WebSockets?

Answer: Raw WebSockets only transmit raw strings or byte arrays; they don't define headers, message types, or routing. STOMP (Simple Text Oriented Messaging Protocol) adds a standardized framing layer (CONNECT, SUBSCRIBE, SEND, MESSAGE). This enables publish/subscribe topic channels (e.g. /topic/room/{name}) so multiple isolated chat rooms can run over one single WebSocket connection.

Q3: How does the system work without internet access?

Answer: The server device runs an embedded Tomcat instance and MySQL locally on the campus Wi-Fi or mobile hotspot. The server machine has a local IP (e.g. 10.5.4.55). Clients connect directly to this local IP via Wi-Fi. All assets (HTML, CSS, JS, SockJS, STOMP libraries) are stored locally in the project JAR, eliminating any external CDN calls.

Q4: How is private channel security enforced?

Answer: Private channels use a 4-digit PIN hashed using **SHA-256**. The database never stores the plaintext PIN. When a user enters a PIN, the backend hashes the input and compares the hashes. Once verified, a ChannelMember record is created in MySQL, granting persistent access.

Q5: How does the dual-tier admin system prevent privilege leaks?

Answer: When a Master Admin or Sub-Channel Admin is removed, the system calls downgradeRoleIfNoAdminRoles() in ChannelService. It checks if the user is still a Master Admin of any other channel (existsByUserAndIsAdminTrue) or Sub-Channel Admin of any other sub-channel (existsByUser). Only if both checks return false is their global User.role revoked back to standard USER.